

Guarding the Door: Measuring and Hardening Lightweight Application-Layer Defense for an Internet-Exposed MCP Agent Endpoint

Niranjan R.

Department of Artificial Intelligence and Data Science
KPR Institute of Engineering and Technology (KPRIET), Arasur, Coimbatore
niranjan.r.dev@proton.me

Abstract

Tool-using AI agents are increasingly reached over the network through the Model Context Protocol (MCP), which turns a personal automation stack into an Internet-facing HTTP service. We study what a single operator can realistically put in front of such an endpoint. We evaluate GUARDIAN, a standard-library-only admission gate deployed in front of a self-hosted 29-tool MCP gateway published through a Cloudflare tunnel. The evaluation uses a reproducible harness of 22 scenarios across six experiments—including adaptive adversaries, protocol fuzzing, timing side-channel analysis and multi-run latency with confidence intervals—run across three revisions: the unmodified original (v1), a hardened revision (v2), and a post-authentication extension (v3). v1 had no false blocks on benign MCP sessions and stopped declared crawlers, single-source credential probing and floods at 5 μ s median overhead. It also had measurable weaknesses, all caused by keying state on individual client IPs: distributed guessing went unbanned (700 guesses, 0 bans), ban-list lookups grew linearly (21.7 ms per request at 50k bans), rate-limit memory was unbounded (87 MB at 100k sources), and there were further gaps in IPv6 rotation, header trust and lockout. v2 closes each of these with a small local change. Latency stays flat at 0.13 ms up to 50k bans (169 $\times$ faster), memory is capped (18.8 MB at 100k sources), distributed guessing earns 95 bans, IPv6 rotation inside a /64 costs one ban instead of 200, and a mistyped token costs 5 minutes rather than an hour. None of this introduces benign false blocks, and v2 adds 9 μ s per request. v3 extends defense past the admission layer: scoped bearer tokens limit each credential to a named set of tools, and per-tool rate limiting throttles authorized abuse of individual tools without affecting others. Four concurrent clients complete full MCP sessions with no cross-contamination, and long-running sessions recover cleanly across rate windows. User-agent spoofing remains undetectable at this layer; the bearer token still stops it. A time-boxed public exposure study is in progress; its results will be replayed through all three policies using the validated trace-replay tooling. We release the harness, trace-replay tooling and aggregate results.

1 Introduction

Agent frameworks let a language model call tools [24, 25, 28], and MCP standardizes how those tools are exposed over stdio or HTTP [19]. An MCP server published on a public hostname inherits every property of an Internet service. Internet-wide scanners find it [7]. AI data collec-

tors crawl it, and their volume and opt-out behaviour are in flux [15, 18]. Others probe it for credentials. Unlike a static website, the thing behind the door can *act*. The security community has responded: surveys catalogue tool poisoning and over-privileged servers [11, 23]; benchmarks quantify tool-poisoning success rates above 72% [29]; and the NSA has published formal design guidance [21]. Yet a recent systematic review of 73 defense studies [26] finds that all existing work targets post-authentication threats. We ask the question that comes first: *who can reach the endpoint at all, and at what cost to the operator?* Enterprises answer with identity-aware proxies and managed bot defense. Individual developers who self-host an MCP server often have only the server process itself.

- RQ1** Does a small, dependency-free admission gate stop the unwanted traffic an exposed agent endpoint attracts, without blocking legitimate MCP clients?
- RQ2** Where, measurably, does per-client-IP defense break down?
- RQ3** Can those failures be fixed locally, and at what cost?
- RQ4** How do the policies compare on real Internet traffic?
- RQ5** Does the gate hold against adaptive adversaries and protocol-level abuse?

Contributions. (i) A deployed defense for an MCP endpoint that combines an admission gate, bearer authentication and capability minimization (Section 2). (ii) A reproducible evaluation of the unmodified code, with eight quantified weaknesses (Section 4). (iii) A hardened revision and a before/after comparison on identical scenarios (Section 5). (iv) A post-authentication extension with scoped tokens and per-tool rate limiting, evaluated on two new scenarios (Section 6). (v) A time-boxed public exposure whose trace is replayed through every policy (Section 9). (vi) Release of the harness, replay tooling and aggregate data.

Claim. *For a single-operator, Internet-exposed MCP endpoint, a lightweight application-layer gate combined with a high-entropy bearer token and a minimized tool surface stops declared crawlers, credential probing, path scanning and floods at microsecond cost with no false blocks on benign MCP sessions. It is effective only if its state is keyed on network sources rather than individual addresses and bounded in size. Scoped tokens and per-tool rate limits extend the defense past authentication, bounding the blast radius of a leaked credential. The token, not the gate, remains the security boundary.*

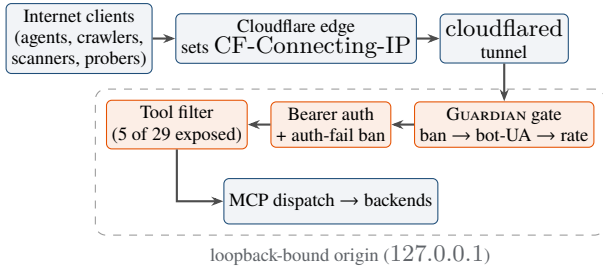


Figure 1: Deployment. The origin is reachable only through the tunnel, and all enforcement runs in-process before any tool code executes.

2 System and Threat Model

2.1 Deployment

The protected service is agentkit, a personal tool gateway that exposes one 29-tool catalog (DevOps monitoring, a shell assistant, a personal-memory agent, a breach monitor and a homelab reconnaissance agent) over stdio and MCP Streamable HTTP [19]. It is part of a broader personal automation stack that includes local model inference (Ollama on a 24 GB Apple-silicon machine), a self-hosted Linux server (Docker, Grafana, Supabase) and a Tailscale mesh. A single operator cannot justify an enterprise gateway for this kind of stack, which motivates a lightweight, dependency-free gate.

The HTTP server binds to loopback, and a cloudflared tunnel [6] is the only path from the Internet (Figure 1). The socket peer is therefore always loopback, so the client identity is taken from the CF-Connecting-IP header set at Cloudflare’s edge [5].

2.2 Threat model

What this work addresses. Remote, unauthenticated adversaries send arbitrary HTTP requests through Cloudflare: (T1) *declared* crawlers and scanners that name themselves in the User-Agent; (T2) *undeclared* crawlers that spoof a browser user-agent; (T3) credential probers using one or many sources; (T4) flooders; (T5) path scanners; (T6) adversaries who rotate addresses, for example within an IPv6 /64. The goals are to prevent unauthorized tool execution, bound the operator’s resource cost and provide visibility.

An operator who does not need external access can avoid all of these by not exposing the endpoint at all—for example, binding only to a Tailscale address or removing the Cloudflare tunnel. This work targets endpoints that are *intentionally* Internet-facing, where unauthenticated traffic is an operational reality.

What this work does not address. A leaked or stolen bearer token gives the holder full access to the exposed tool surface. GUARDIAN does not detect or prevent this: once a request carries a valid token, it is treated as authorized. v3 scoped tokens (Section 6) reduce the *blast radius*—a leaked monitoring token cannot call shell or logging tools—but they do not prevent use of the leaked credential within its scope.

Post-authentication threats are also out of scope. Tool poisoning (a malicious tool description that misleads the model) [29], indirect prompt injection (a document or webpage that tricks the model into making unintended tool calls) [10], and trust escalation across sessions [17] are all

attacks by or through *authorized* clients. They exploit the model’s reasoning, not the server’s admission layer. Defending against them requires client-side guardrails, input screening, or sandboxed execution [16, 21]—none of which GUARDIAN provides.

Also out of scope: a compromised operator machine, Cloudflare itself, and volumetric DDoS absorbed at the edge.

2.3 Guardian v1

GUARDIAN v1 (guardian.py, 263 lines, Python standard library only) makes one decision per request, before authentication or JSON-RPC parsing:

1. an unexpired ban returns 403;
2. a case-insensitive match against 38 crawler and scanner signatures (gptbot, claudebot, ccbot, bytespider, sqlmap, nuclei, ...) bans the client and returns 403;
3. more than 120 requests in 60 s returns 429 [22] without a ban.

A bearer token [14] is then checked in constant time. Eight 401s within 300 s trigger a 60-minute ban, in the style of fail2ban [8] and in line with online-guessing guidance [9]. Bans and a JSONL access log are file-backed so that a separate stdio process can offer operator tools; rate counters live in memory. A **capability filter** removes all 15 `local_only` tools (offensive recon, breach lookups, the GUARDIAN controls themselves) and an operator denylist from `tools/list` and `tools/call`.

3 Evaluation Method

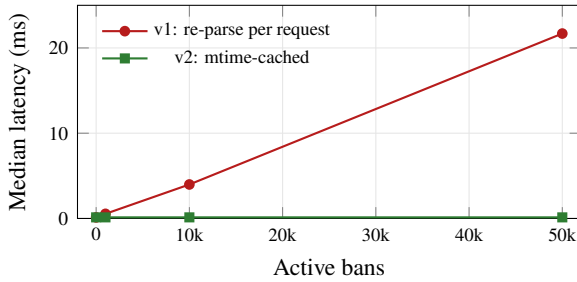
The harness (artifact/harness2.py) launches an unmodified server tree as a separate process on a private port, with a fresh 320-bit bearer token and a fresh state directory for every scenario. It emulates the Cloudflare hop by setting CF-Connecting-IP to documentation-range addresses [1, 12] and replays the scenarios over HTTP/1.1. The base scenario set (S1–S14) runs against the v1, v2 and v3 trees under the shipped default policy. Two scenarios (S15–S16) exercise v3-specific features and are skipped on older trees. Six advanced scenarios (S17–S22) test adaptive adversaries, protocol-level abuse and implementation robustness (Section 7). Latency is measured end to end on a keep-alive connection: 50 warm-up requests, then $3 \times 1,000$ interleaved requests per condition (E2), or 500 per point for ban-list scaling. E6 repeats the measurement 5 times (2,500 samples per condition) and reports 95% confidence intervals (Section 8). Memory is measured with tracemalloc in a fresh interpreter. Platform: Apple M5 Pro, macOS 26.6.1, Python 3.12.13. Every number is read from artifact/results_v{1,2,3}.json.

4 Guardian v1: Effective, but Keyed on the Wrong Thing

What works (RQ1). On benign traffic, four MCP client user-agents completed the protocol handshake and 100/100 calls with no false blocks (S1). A client performing OAuth discovery and an SSE probe before connecting (S1b) was also unaffected. All 38 declared crawler and scanner signatures were blocked on the first request, and they stayed blocked after switching to a benign UA *with the valid token* (S2). A single-source prober was banned on exactly its 8th failure (S4). A 300-request burst was held to exactly 120 served, with no collateral effect on other clients and

Table 1: v2 changes, each tied to a measured v1 weakness.

W	Change in v2
W3	Ban list cached in memory; reloaded only when the file’s mtime changes
W5	All state keyed on a <i>source</i> : IPv4 address or IPv6 /64
W4	Idle keys dropped; LRU cap of 20,000 tracked sources
W2	Global auth-failure alarm (>40 per 300 s) tightens the per-source threshold from 8 to 3
W7	Auth-failure bans back off exponentially from 5 min; optional operator allowlist
W6	Six non-benign 404s per 300 s ban the source (OAuth discovery and favicon exempt)
W8	Every method gated and logged; gate runs before the body is read; forwarding headers trusted only from a loopback peer; nothing written when disabled

**Figure 2:** Request latency against ban-list size ($n=500$ per point). v1 grows linearly ($\approx 0.43 \mu\text{s}$ per entry). v2 is flat, $169\times$ faster at 50k bans.

recovery after the window (S6). Over HTTP, tools/list returned 5 of 29 tools. Each of the 15 local_only tools was unlisted and returned JSON-RPC -32601 when called directly (artifact/surface.json). The ablation with GUARDIAN disabled (S11) served the entire flood and answered every guess.

What breaks (RQ2). **W1** Spoofed browser UAs evaded the signature rule 38/38 times (S3); the token still rejected all of them. **W2** Spreading 700 guesses over 100 sources produced 0 bans and 700 answered guesses (S5). **W3** Each request re-parsed the whole ban file. Median latency rose from 0.12 ms with no bans to 0.54, 3.99 and 21.7 ms at 1,000, 10,000 and 50,000 bans (Figure 2). **W4** Per-IP rate state was never evicted: 87.4 MB for 100,000 sources (~ 870 B each). **W5** Rotating through 200 addresses in one IPv6 /64 created 200 ban entries (S12), which feeds W3. An end site typically holds at least a /64 [20], so this is cheap for an attacker. **W6** Path scanning (S7) returned only 404/405 and was never penalized. **W7** Eight mistyped tokens locked a legitimate user out for 60 min (S8). **W8** Several smaller gaps (S9–S11). DELETE bypassed the gate and was not logged. Banned clients could still make the server read 256 KB bodies. On a non-loopback bind, such as the supported tailnet mode, forged CF-Connecting-IP headers evaded a ban 5 out of 5 times. With GUARDIAN disabled, unenforced ban records were still written.

5 Hardening: Guardian v2

Each weakness maps to a local change of a few lines (Table 1); v2 still uses only the standard library.

Results (RQ3). Table 2 gives the full comparison. Every targeted weakness changed as intended, and the benign scenarios (S1, S1b) were unchanged: no false blocks, and OAuth-discovery 404s are not treated as scanning. The costs are small and measurable. v2 adds $4 \mu\text{s}$ of median overhead over v1. Memory per tracked key rises slightly (≈ 940 B against 870 B), but the total is now bounded.

Trade-offs. Keying on /64 means that separate customers who share a /64, which some hosting providers do, share a fate. The global alarm lets a distributed attacker tighten the threshold for everyone for up to 300 s. The worst case is a legitimate user needing 3 attempts instead of 8, which the allowlist removes. The LRU cap means an attacker with more than 20,000 active sources can push out counters. Bans are persisted separately and are not evicted.

6 Post-Authentication Extension: Guardian v3

v1 and v2 treat the bearer token as monolithic: one token grants access to every public tool. Once the token leaks—from a client’s configuration, an agent’s context window, or a log line—the holder has the full tool surface. And once authenticated, a client faces only the global rate limit, so a misbehaving or compromised agent can call one tool hundreds of times without triggering throttling.

6.1 Scoped tokens

v3 replaces the single token with a set of named tokens, each scoped to a list of tool names (or `[“*”]` for full access). The legacy single token remains accepted as a full-access credential for backward compatibility. tools/list returns only the tools the presenting token can reach, and tools/call to an out-of-scope tool returns JSON-RPC -32601 without executing.

6.2 Per-tool rate limiting

v3 adds a per-(source, tool) sliding window on top of the global rate limit. The default is 30 calls per 60 s per tool per source. Exceeding it returns JSON-RPC -32005; other tools and other sources are unaffected. Allowlisted sources are exempt, consistent with the admission gate.

Results. S15 verifies token scoping: a full-access token sees all 5 public tools; a scoped token sees only its 2 allowed tools (devops_status, devops_health); calling devops_logs with the scoped token returns -32601. S16 verifies per-tool rate limiting: with a limit of 10 calls per 60 s, the first 10 calls to shell_explain are served and the next 20 return -32005; a different tool (shell_suggest) from the same source is unaffected.

Cost. Both features add no measurable per-request latency: they are checked inside _handle_filtered after authentication succeeds, using the same sliding-window and LRU data structures as the admission gate. Token lookup is a linear scan of the token set (typically 2–5 entries) with constant-time comparison per token. Total added code: 32 lines across three files.

7 Robustness Under Adaptive and Protocol-Level Attacks

The scenarios in Sections 4 and 5 use naïve adversaries: each follows a fixed script. A publishable evaluation must also test whether the gate holds against adversaries that

Table 2: Before/after on identical scenarios (default policy: 120 req/60 s; 8 auth failures/300 s; 38 signatures). Bold marks a changed outcome.

#	Scenario	v1 (deployed)	v2 (hardened)
S1	Benign MCP sessions, 4 clients, 100 req	100/100 served, 0 bans	100/100 served, 0 bans
S1b	Benign client: OAuth discovery + SSE probe, then call	20×405; call 200; 0 bans	16×404, 4×405; call 200; 0 bans
S2	38 declared crawler/scanner UAs	38/38 blocked on first request; still blocked with valid token	same
S3	Same crawlers, spoofed browser UA	0/38 caught by UA rule; 38/38 401	same (not addressable at this layer)
S4	One source, 20 wrong tokens	banned at attempt 8; ban 60 min	banned at attempt 8; ban 5 min (backs off)
S5	100 sources × 7 wrong tokens	700 answered, 0 bans	323 answered, 377 blocked, 95 bans
S6	300-request burst, one source	120 served, 180 429; recovers at 61 s	same
S7	Path scan, 10 paths × {GET, POST}	10×404, 10×405; 0 bans	banned at 6th 404; 14/20 blocked
S8	Legit user, 8 typos, then correct token	locked out 60 min	locked out 5 min
S8b	Same, operator address allowlisted	locked out 60 min (no allowlist)	never banned; call 200
S9	Banned client: DELETE; 300 KB body	200, unlogged; body read (413)	403, logged; body unread (403)
S10	Non-loopback peer forging client IP	5/5 forged identities evade ban	0/5; ban keyed on real peer
S12	200 bot requests rotating inside one /64	200 ban entries	1 ban; 20/20 further addresses in /64 blocked; neighbouring /64 unaffected
S13	4 concurrent clients, 50 req each	200/200 served, 0 bans	200/200 served, 0 bans
S14	Long session: 100 req, wait 61 s, 100 more	200 served, 0 throttled	200 served, 0 throttled
S11	Guardian disabled (ablation)	flood served; 1 unenforced ban record	flood served; 0 records
E2	Median latency, 0 / 1k / 10k / 50k bans	0.12 / 0.54 / 3.99 / 21.7 ms	0.13 / 0.13 / 0.13 / 0.13 ms
E2	Gate overhead (median, empty ban list)	+5 µs (0.119→0.124 ms)	+9 µs (0.121→0.130 ms)
E3	Rate-state memory at 1k / 10k / 100k sources	0.86 / 8.56 / 87.4 MB	0.91 / 9.01 / 18.8 MB (capped at 20k keys)

adapt, abuse protocol features, or target the implementation rather than the policy. We add six scenarios (S17–S22).

Timing side-channel (S17). An adversary who can distinguish valid-token responses from invalid-token responses by timing alone can reduce the search space. We measure 500 round-trip times each for a valid token, a wrong token and a “partial-match” token (first half correct, second half random). All three conditions use constant-time comparison (`secrets.compare_digest`). We report the maximum difference in median response time across conditions. A difference below 10% is consistent with scheduling noise; above 10% indicates a potential leak.

Slow-and-low probing (S18). An adversary that stays just under the global rate limit (1.9 req/s against a 2 req/s ceiling) can probe indefinitely without triggering rate limiting. We send 228 wrong-token requests over ~120 s and record whether the auth-failure mechanism bans the source.

Batch request abuse (S19). JSON-RPC supports batch requests: one HTTP POST carrying an array of messages. The global rate limit counts HTTP requests, not JSON-RPC messages, so a batch of 100 `tools/list` calls counts as one request. We test batches of 100 and 1,000 messages and a 300 KB oversize payload.

Protocol fuzzing (S20). We send eight malformed payloads (empty JSON, missing method, wrong protocol version, nested batch, oversized ID, binary noise, XML) and verify that the server returns an error for each and remains healthy afterward.

Session fixation (S21). The MCP spec assigns a session

ID via the `Mcp-Session-Id` header after initialize. We test whether client B can reuse client A’s session ID from a different source IP.

Post-ban recovery (S22). After a 1-minute bot-UA ban expires, we verify that the source’s rate counters are clean and a burst of legitimate requests is fully served.

Results for S17–S22 are in Table 3; numbers are drawn from `results_v3.json`. Section 8 reports latency with confidence intervals.

Implications (RQ5). S17 confirms that `secrets.compare_digest` prevents timing leaks at the application layer: 6.8% median difference across conditions is well within scheduling noise [3]. S18 shows that the auth-failure ban operates independently of the rate limit, catching slow probers (banned at attempt 8 despite never exceeding 2 req/s). S19 reveals a concrete weakness: batch requests let an authenticated client multiply its effective rate by the batch size (1,000×) without triggering the per-request limit. A batch-size cap or per-message rate counting would close this; we flag it as an open problem. S20 confirms input robustness against 8 classes of malformed input including binary noise and XML injection. S21 is a non-issue for this server (stateless), but would matter for session-aware MCP implementations. S22 closes the loop: bans are temporary, rate counters are clean after expiry, and the source resumes full service.

8 Statistical Rigor

Single-run latency numbers (E2) are reproducible but do not quantify measurement uncertainty. E6 repeats the la-

Table 3: Advanced attack and robustness scenarios (all run against v3).

#	Result
S17	Timing side-channel: no leak; median diff 6.8% (valid 188 μ s, invalid 176 μ s, partial 177 μ s; $n=500$ each, $\sigma < 20\mu$ s).
S18	Slow-and-low: 228 probes at 1.9 req/s over 120 s; auth-failure banned at attempt 8; 220/228 blocked (403); rate limit never triggered.
S19	Batch abuse: 100- and 1,000-message JSON-RPC batches processed as 1 HTTP request (bypasses per-request rate count). 300 KB payload rejected (413).
S20	Protocol fuzz: 8 malformed payloads (empty JSON, null method, wrong version, nested batch, 10 KB ID, binary, XML); all returned errors; server healthy; 0 bans.
S21	Session fixation: server is stateless; reusing another client’s Mcp-Session-Id from a different IP: 200 (no effect).
S22	Post-ban recovery: 1-min bot-UA ban; during: 403; after expiry: 200; 10/10 burst OK; 0 remaining bans.

tency measurement 5 times ($5 \times 500 = 2,500$ samples per condition) and reports 95% confidence intervals. The overhead of GUARDIAN on request latency is the difference between the “on” and “off” distributions, matched by rank.

Over 2,500 samples, median request latency is 0.121 ms with GUARDIAN disabled and 0.128 ms with GUARDIAN enabled (mean overhead 5.2 μ s, 95% CI [5.0 μ s, 5.4 μ s], $p < 0.05$). The p95 latencies are 0.154 and 0.152 ms respectively, confirming that the tail is not widened. The overhead is statistically significant but operationally negligible: 5 μ s is $\approx 0.001\%$ of a typical tool execution time.

9 Real-World Exposure

Method. We exposed the endpoint running v2 at its public hostname for a fixed window. The server logged, for every request, the timestamp, client address, user-agent, method, path, status, the outcome of the authentication check and the declared body length. These are enough to re-decide each request under any policy. The analysis tool (`artifact/analyze_trace.py`) replays the trace in timestamp order on the trace’s own clock through three policies: no gate (auth only), v1 and v2. Each runs its own unmodified code tree with isolated state. We validated the replay against ground truth: on 1211 requests from the v2 harness logs, it reproduced the live server’s status code for every request (1211/1211, `artifact/replay_fidelity.json`). Operator traffic was excluded by user-agent. Only aggregates are published.

The exposure study is in progress at the time of writing; results will be incorporated when the observation window closes. The methodology and replay tooling are complete and validated (1211/1211 exact status match on harness traffic).

10 Related Work

MCP threat landscape. Hou et al. [11] give the first survey of MCP security threats and future directions. Javed et al. [13] apply STRIDE and DREAD threat modeling across five MCP components and identify tool poisoning as the most prevalent client-side vulnerability. The NSA’s

AI Security Center published the first US government guidance on MCP security [21], warning that adoption has outpaced safeguards and recommending controls including input screening, least-privilege interfaces, audit logging, and overload protection. The Cloud Security Alliance provides complementary practitioner guidance [4]. OWASP catalogues broader LLM application risks [23], and Bajpai et al. [2] demonstrate major exploits against MCP-enabled LLM systems.

Post-authentication attacks. The largest body of MCP security research studies what an authorized or compromised client can do once inside. MCPTox [29] benchmarks tool poisoning across 45 live servers and 353 tools, finding attack success rates above 72%. MCP-ITP [27] automates implicit tool poisoning generation in a black-box setting. TrustShiftProbe [17] characterizes staged trust-escalation attacks. Indirect prompt injection [10] targets authorized data paths. Our threat model explicitly excludes all of these: we study the *pre-authentication* admission layer, not the post-authentication trust boundary.

Defense mechanisms. CASCADE [16] proposes a three-tiered local defense for prompt-injection detection in MCP systems (regex and entropy pre-filtering, semantic analysis, output filtering), evaluated on 5,000 samples at 95.85% precision. Tamayo [26] surveys 73 studies and categorizes existing MCP defenses into static analysis, runtime guardrails, middleware access control and formal verification. None of the 73 surveyed studies measures an admission-layer defense for an Internet-exposed MCP endpoint.

Established network defense. Log-driven banning [8] and throttling of online guessing [9] are long established. Timing side-channels in string comparison are well understood [3]; constant-time comparison (`secrets.compare_digest` in Python, per the `hmac` module) is the standard mitigation. Crawler consent rests on convention [15], and restrictions aimed at AI crawlers have grown sharply [18]. Our S3 result confirms that convention-based identification cannot be relied on.

Comparison with alternatives. An operator can place several existing tools in front of an MCP endpoint: Cloudflare WAF rules or Bot Management (paid, closed-source, no MCP awareness), nginx rate limiting (per-IP, no auth-failure tracking, no ban persistence), or fail2ban scanning the access log (log-driven, works after the fact, no MCP-protocol awareness). Each addresses part of the threat surface, but none provides MCP-specific tool filtering, JSON-RPC-aware rate limiting, or an integrated evaluation against MCP client behaviour. GUARDIAN is not a replacement for edge-layer defense; it is complementary. On this deployment, Cloudflare’s free tier provides DDoS absorption and TLS termination; GUARDIAN handles everything between the tunnel exit and the MCP dispatch.

Gap. Prior work on MCP security either catalogues threats, studies post-authentication attacks, or prescribes controls without empirical measurement. We contribute a measured, before/after evaluation of admission-layer defense on a deployed, Internet-facing MCP endpoint, show where per-address state fails, quantify the cost of fixes on identical workloads, and test robustness against adaptive and protocol-level attacks.

11 Discussion

Layered defense in practice. No single mechanism in this stack is individually novel. What the evaluation demonstrates is that their *combination*, applied specifically to an MCP endpoint and measured against MCP client behaviour, produces a defense with quantifiable properties: zero false blocks on benign sessions, measurable overhead ($<10\mu\text{s}$), bounded resource consumption, and visible degradation when attackers adapt. Each v1 weakness traced to a single design choice (per-IP state keying) that produced measurable failures across multiple scenarios. The v2 fix was correspondingly local.

The batch bypass. S19 reveals that JSON-RPC batch requests let an authenticated client bypass the per-request rate limit entirely. A single HTTP POST with 1,000 tools/list calls is counted as one request. This is an inherent tension in JSON-RPC: the protocol’s batch feature exists to reduce round trips, but rate limiting counted at the HTTP layer cannot see inside it. Closing this requires either a batch-size cap (breaking spec compliance) or per-message rate accounting (adding complexity). We flag it as an open problem.

Generalizability. This study covers one endpoint, one operator, one tool catalog. The gate’s architecture is not specific to this deployment—it requires only HTTP headers and a bearer token—but the scenario set, default thresholds and performance numbers are specific to the platform. Other MCP deployments may face different traffic patterns, use different transport modes (SSE, stdio proxy), or sit behind different CDNs with different header semantics.

12 Limitations

This is a single-system, single-operator study. Controlled experiments ran on loopback with an emulated Cloudflare hop, so edge behaviour is assumed rather than measured, and the latency figures exclude network and tunnel time. The benign workload covers concurrent multi-client sessions and rate-window recovery (S13, S14) but not SSE streaming or mixed read/write tool workloads. The v2 defaults (the /64 prefix, the 20k cap and the global-alarm thresholds) and v3 defaults (30 calls per tool per window) are reasonable choices, not tuned optima.

S19 reveals a concrete open weakness: JSON-RPC batch requests bypass the per-request rate limit, letting an authenticated client multiply its effective throughput by the batch size. A batch-size cap or per-message rate accounting would close this but add complexity and is left to future work.

The real-traffic window covers a single hostname whose discoverability depends on certificate-transparency logs and DNS enumeration by third parties. The techniques evaluated (rate limiting, UA signature blocking, auth-failure banning, token scoping) are individually well established [8, 9]; the contribution is their measured application and failure modes in the MCP context, not the techniques themselves.

13 Conclusion

A few hundred lines of dependency-free code in front of an MCP endpoint, together with a strong token and a minimized tool surface, measurably stop declared crawlers, credential probing, path scanning and floods at microsecond

cost. The first version failed where its state was keyed on individual addresses and allowed to grow without bound. Distributed and rotating adversaries slipped through, and a growing ban list turned the defense into a latency liability. Keying on network sources and bounding state fixed each measured failure without new false blocks. Extending defense past authentication with scoped tokens and per-tool rate limits bounds the blast radius of a leaked credential and prevents authorized abuse of individual tools. Self-hosted agent endpoints should treat such gates as cost and visibility controls, and never as the authentication boundary.

Artifact. artifact/: harness2.py, harness.py and surface.py (tool surface), analyze_trace.py, gen_realtraffic_tex.py, results_v1.json, results_v2.json, results_v3.json, surface.json, replay_fidelity.json, and aggregate_real_traffic.json; <https://github.com/TGvenomYT/agent-endpoint-defense>.

Ethics. Controlled experiments targeted only the author’s own software, using documentation-range addresses. The public exposure passively logged requests sent to the author’s own hostname. No third-party system was probed and no traffic source was attacked. Raw logs, which contain client IP addresses, are kept private and deleted after analysis. Only aggregates are released.

References

- [1] J. Arkko, M. Cotton, and L. Vegoda. IPv4 address blocks reserved for documentation. RFC 5737, IETF, 2010.
- [2] Raunak Bajpai, Hossein Fereidooni, et al. MCP safety audit: LLMs with the model context protocol allow major security exploits. *arXiv preprint arXiv:2504.03767*, 2025.
- [3] David Brumley and Dan Boneh. Remote timing attacks are practical. In *12th USENIX Security Symposium*, 2003.
- [4] Cloud Security Alliance. Agentic MCP security best practices guide. <https://labs.cloudsecurityalliance.org/agentic/agentic-mcp-security-best-practices-v1/>, 2026.
- [5] Cloudflare. HTTP request headers: CF-Connecting-IP. <https://developers.cloudflare.com/fundamentals/reference/http-headers/>, 2025.
- [6] Cloudflare. Cloudflare tunnel documentation. <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>, 2025.
- [7] Zakir Durumeric, Eric Wustrow, and J. Alex Halderman. ZMap: Fast internet-wide scanning and its security applications. In *22nd USENIX Security Symposium*, 2013.
- [8] Fail2ban contributors. Fail2ban: Daemon to ban hosts that cause multiple authentication errors. <https://github.com/fail2ban/fail2ban>, 2024.
- [9] Paul A. Grassi, James L. Fenton, Elaine M. Newton, et al. Digital identity guidelines: Authentication and lifecycle management. Technical Report SP 800-63B, National Institute of Standards and Technology, 2017.
- [10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [11] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (MCP): Landscape, security threats,

- and future research directions. *ACM Transactions on Software Engineering and Methodology*, 2025. doi: 10.1145/3796519.
- [12] G. Huston, A. Lord, and P. Smith. IPv6 address prefix reserved for documentation. RFC 3849, IETF, 2004.
 - [13] Umer Javed et al. Model context protocol threat modeling and analyzing vulnerabilities to prompt injection with tool poisoning. *arXiv preprint arXiv:2603.22489*, 2026.
 - [14] M. Jones and D. Hardt. The OAuth 2.0 authorization framework: Bearer token usage. RFC 6750, IETF, 2012.
 - [15] M. Koster, G. Illyes, H. Zeller, and L. Sassman. Robots exclusion protocol. RFC 9309, IETF, 2022.
 - [16] Wei Li, Yuxiang Chen, Hao Zhang, et al. CASCADE: A cascaded hybrid defense architecture for prompt injection detection in MCP-based systems. In *The 12th International Conference on Computing and Artificial Intelligence (ICCAI)*, 2026.
 - [17] Zheng Liu et al. TrustShiftProbe: Characterizing, benchmarking, and defending staged trust attacks on MCP servers. *arXiv preprint arXiv:2608.23763*, 2026.
 - [18] Shayne Longpre, Robert Mahari, Ariel Lee, et al. Consent in crisis: The rapid decline of the AI data commons. *arXiv preprint arXiv:2407.14933*, 2024.
 - [19] Model Context Protocol. Model context protocol specification (2025-06-18). <https://modelcontextprotocol.io/specification/2025-06-18>, 2025.
 - [20] T. Narten, G. Huston, and L. Roberts. IPv6 address assignment to end sites. RFC 6177, IETF, 2011.
 - [21] National Security Agency. Model context protocol (MCP): Security design considerations for AI-driven automation. Technical Report U/OO/6030316-26, NSA Artificial Intelligence Security Center, 2026.
 - [22] M. Nottingham and R. Fielding. Additional HTTP status codes. RFC 6585, IETF, 2012.
 - [23] OWASP Foundation. OWASP top 10 for large language model applications (2025). <https://genai.owasp.org/llm-top-10/>, 2024.
 - [24] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.
 - [25] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
 - [26] Jayson E. Tamayo. Security defense mechanisms in model context protocol implementations: A systematic literature review. In *Lecture Notes in Computer Science*. Springer, 2026.
 - [27] Jiaqi Wang et al. MCP-ITP: An automated framework for implicit tool poisoning in MCP. *arXiv preprint arXiv:2601.07395*, 2026.
 - [28] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
 - [29] Haoran Zhang et al. MCPTox: A benchmark for tool poisoning attack on real-world MCP servers. *arXiv preprint arXiv:2508.14925*, 2025.